EGCO 425 – Chapter 4

Classification Methods

- Decision Trees
- Rule Induction

References

[1] Tan et al., Introduction to Data Mining (Chapters 4-5)

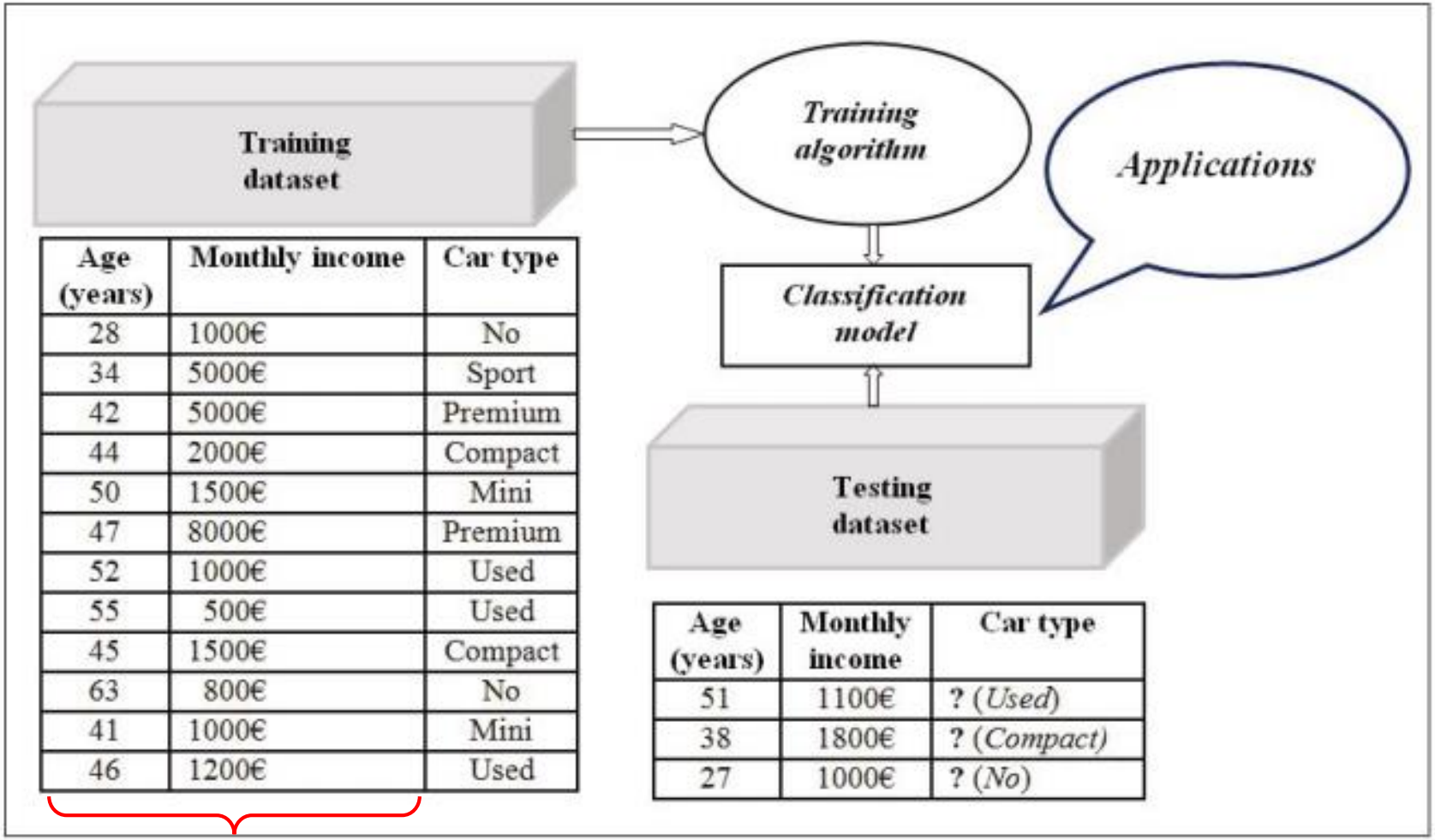
[2] Han & Kamber, Data Mining: Concepts and Techniques (Chapter 8)

[3] Kotu & Deshpande, Data Science: Concepts and Practice (Chapter 4)

[4] RapidMiner Doc, https://docs.rapidminer.com/latest/studio/operators/

R. Marukatat

Classification



Age (years)	Monthly income	Car type
28	1000€	No
34	5000€	Sport
42	5000€	Premium
44	2000€	Compact
50	1500€	Mini
47	8000€	Premium
52	1000€	Used
55	500€	Used
45	1500€	Compact
63	800€	No
41	1000€	Mini
46	1200€	Used

Age (years)	Monthly income	Car type
51	1100€	? (*Used*)
38	1800€	? (*Compact*)
27	1000€	? (*No*)

Attributes Class

- Classification is a supervised learning task because class labels are given
- Learning phase ➔ build a model from training records
  - Classification model = a function of attributes (independent vars) to predict class labels (dependent var)
- Prediction phase ➔ apply the model to new data
  - Testing records with known labels for performance evaluation
  - Unseen records with unknown labels
- Most common & important task in data mining

Some learning techniques

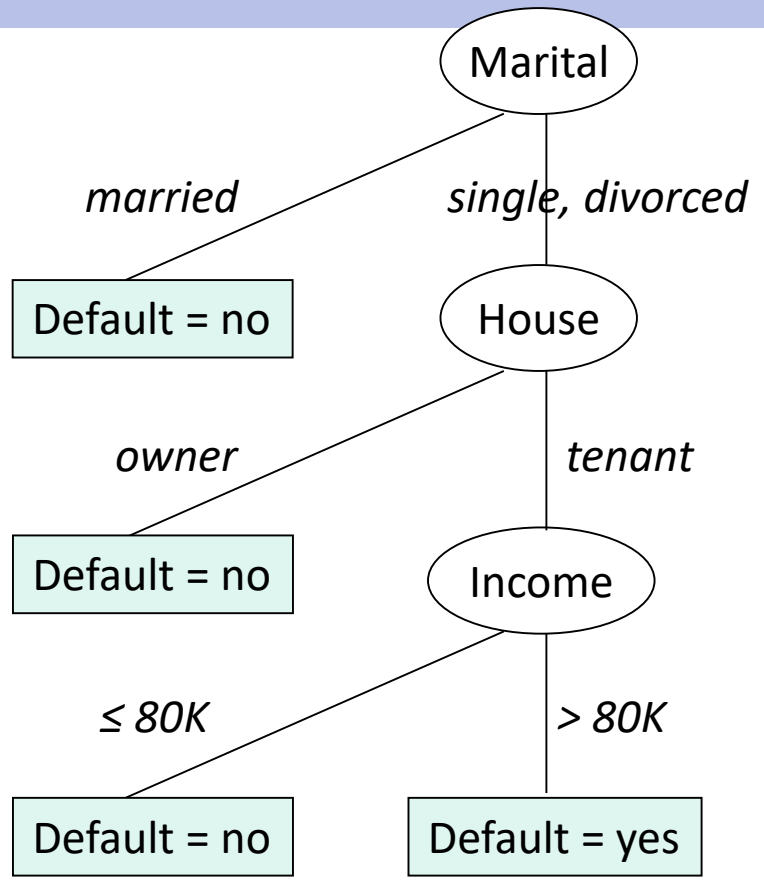
- Decision models
  - Decision trees — ID3, C4.5, CHAID
  - Rules — OneR, RIPPER
- Math models
  - Hyperplane — support vector machine (SVM)
  - Probability — Naïve Bayes
  - Regression — logistic regression
- Others
  - Lazy (no model) — KNN
  - Neural network

Chapter Outline

- Decision tree
  - Tree construction : Hunt's algorithm
  - Attribute selection : Gini, entropy, information gain, gain ratio
  - Pre-pruning vs. post-pruning
- Rule induction
  - Properties of rule set & resolution : exhaustive, mutually exclusive
  - OneR (single rule induction)
  - Sequential covering & rule growing

Decision Tree

TID	House	Marital Status	Income	Default
1	owner	single	125K	no
2	tenant	married	100K	no
3	tenant	single	70K	no
4	owner	married	120K	no
5	tenant	divorced	95K	yes
6	tenant	married	60K	no
7	owner	divorced	220K	no
8	tenant	single	85K	yes
9	tenant	married	75K	no
10	tenant	single	90K	yes



New case
house = tenant, marital = single,
income = 85K → default = ?

Hunt's Algorithm (to build a tree)

Starting at root, with all training records

Loop {

1. Consider remaining data at root ➔ choose 1 attribute that best splits the records to be the tree node (i.e. root of subtree)
2. Split the records by the chosen attribute into K child nodes
3. If a child node contains records from >1 classes ➔ further split to get purer partitions

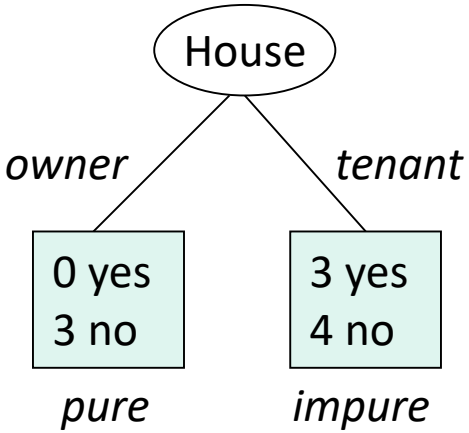
}

If a leaf node is not pure, prediction = majority class in that node

- Points to consider
  - How to determine the best split ?
  - When to stop growing tree ?

Determining the best split

- Goal = each node contains records from the same class
  - Criteria = impurity measure e.g. **Gini**, **entropy**, **information gain**
  - Good split ➔ pure bins
    - Low Gini or low entropy
    - High information gain
      = (Gini or entropy before split) – (Gini or entropy after split)



- Pick the best split from
  - Split by house {owner, tenant}
  - Split by marital {single, married, divorce} {single+married, divorce}
    {single+divorce, married} {single, married+divorce}
  - Split by income best splits from entropy-based discretization

- Measurement at each child node or each bin
  - t = each child node or each bin
  - Pc = proportion of class c in node t
  - **Gini(node t)** $= 1 - \sum_{all\ classes} (Pc)^2$
  - **Entropy(node t)** $= -\sum_{all\ classes} (Pc)\log_2(Pc)$

In previous slide we have P_{yes} , P_{no} in each child node

- Measurement of each split into n child nodes
  - Total Gini $= \sum_{all\ child\ nodes} relative\ size\ of\ t \times Gini(t)$
  - Total entropy $= \sum_{all\ child\ nodes} relative\ size\ of\ t \times entropy(t)$

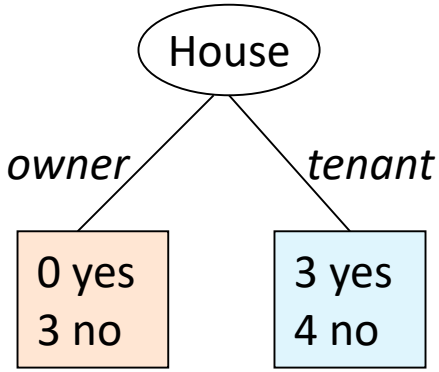
Example : Gini and entropy calculation

At root node, the whole data set has 3 yes + 7 no

Gini(root) = $1 - (3/10)^2 - (7/10)^2$ = 0.42

Entropy(root) = $- 3/10 \log (3/10) - 7/10 \log (7/10)$ = 0.8813

Split by house : {owner, tenant}



Gini(house) = $0.3 * (1 - (0/3)^2 - (3/3)^2) + 0.7 * (1 - (3/7)^2 - (4/7)^2)$

= 0.343

Entropy(house) = $0.3 * (- 0/3 \log (0/3) - 3/3 \log (3/3)) + 0.7 * (- 3/7 \log (3/7) - 4/7 \log (4/7))$

= 0.6897

Information gain (Gini) = 0.42 - 0.343 = 0.077

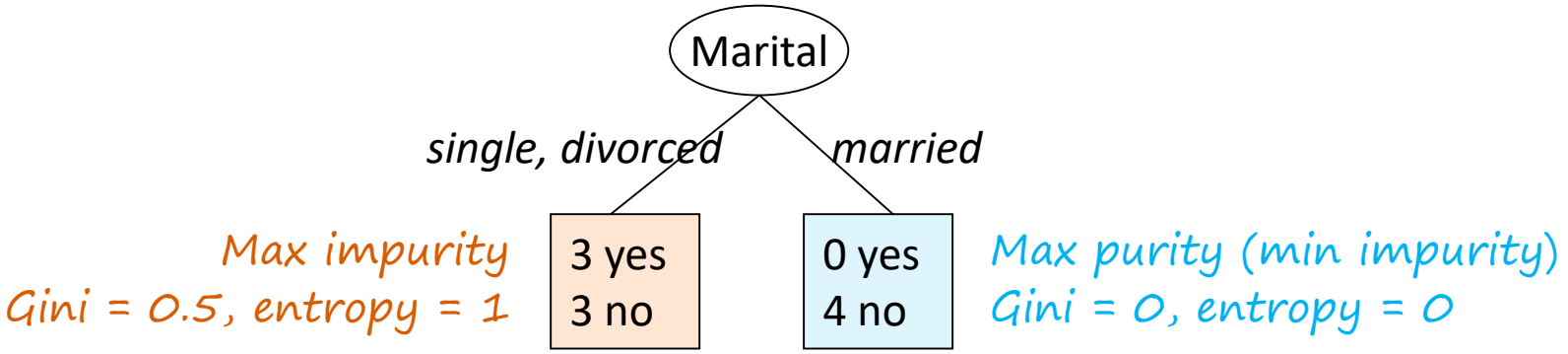
Information gain (entropy) = 0.8813 - 0.6897 = 0.1916

Split by marital status (binary) : {single+divorce, married}

Gini(marital) = $0.6 * (1 - (3/6)^2 - (3/6)^2) + 0.4 * (1 - (0/4)^2 - (4/4)^2)$
= <u>0.3</u>

Entropy(marital) = $0.6 * (- 3/6 \log (3/6) - 3/6 \log (3/6)) +$
$0.4 * (- 0/4 \log (0/4) - 4/4 \log (4/4))$
= <u>0.6</u>

Information gain (Gini) = 0.42 - 0.3 = <u>0.12</u>
Information gain (entropy) = 0.8813 - 0.6 = <u>0.2813</u>



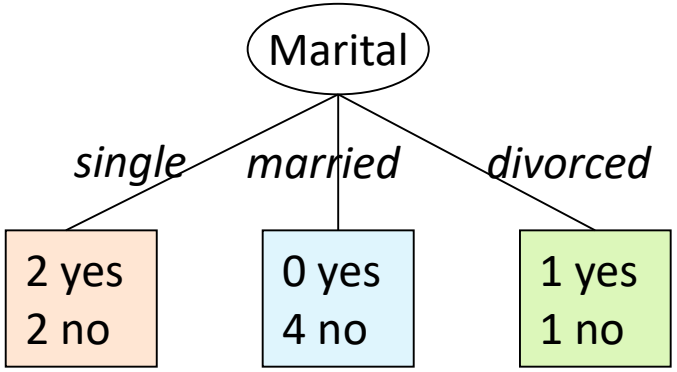
Split by marital status (multiway) : {single, married, divorced}

Gini(marital) = $0.4 * (1 - (2/4)^2 - (2/4)^2) + 0.4 * (1 - (0/4)^2 - (4/4)^2)$
$+ 0.2 * (1 - (1/2)^2 - (1/2)^2)$ = 0.3

Entropy(marital) = $0.4 * (- 2/4 \log (2/4) - 2/4 \log (2/4)) +$
$0.4 * (- 0/4 \log (0/4) - 4/4 \log (4/4))$
$0.2 * (- 1/2 \log (1/2) - 1/2 \log (1/2))$ = 0.6

Information gain (Gini) = 0.42 - 0.3 = 0.12

Information gain (entropy) = 0.8813 - 0.6 = 0.2813

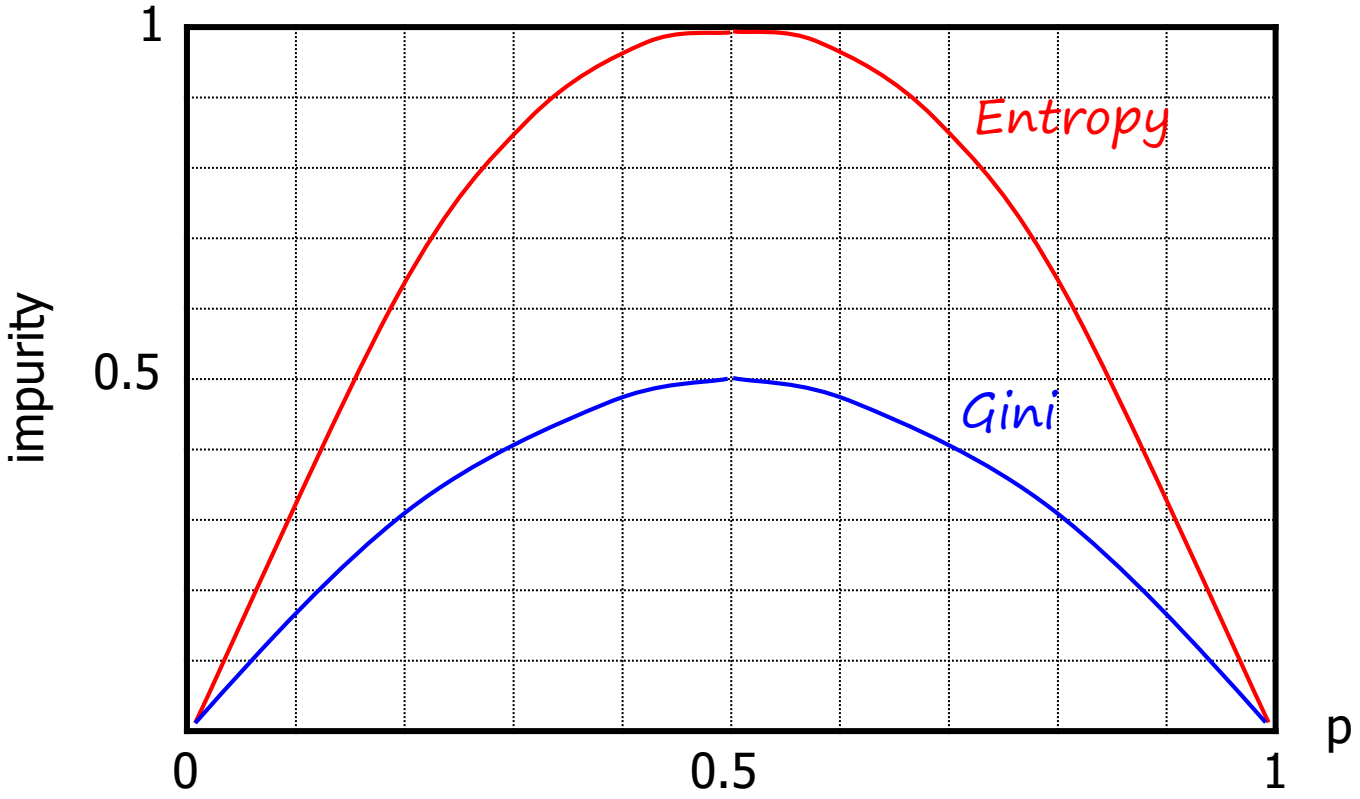


Split by income : entropy-based discretization

Class	No	No	No	Yes	Yes	Yes	No	No	No	No
Income (sorted)	60	70	75	85	90	95	100	120	125	220
Split (1) = 65	0 y 1 n	3 y 6 n	total Gini = $(0.1 * 0) + 0.9 * (1 - (3/9)^2 - (6/9)^2) = 0.4$							
Split (2) = 72	0 y, 2 n		3 y, 5 n	total Gini = 0.375						
Split (3) = 80	0 y, 3 n			3 y, 4 n	total Gini = 0.343					
Split (4) = 87	1 y, 3 n				2 y, 4 n	total Gini = 0.417				
Split (5) = 92	2 y, 3 n	total Gini = 0.4				1 y, 4 n				
Split (6) = 97	**3 y, 3 n**	**total Gini = 0.3** ✓					**0 y, 4 n**			
Split (7) = 110	3 y, 4 n	total Gini = 0.343						0 y, 3 n		
Split (8) = 122	3 y, 5 n	total Gini = 0.375							0 y, 2 n	
Split (9) = 172	3 y, 6 n	total Gini = 0.4								0 y 1 n

Gini vs. Entropy

- Different scales but same behavior
- 2-class problem with class distribution = (p, 1-p)
- For multi-class problem, Gini may > 0.5 and entropy may > 1



Split Penalty

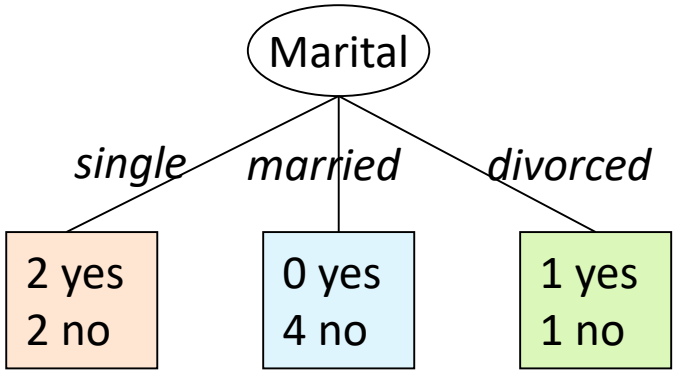
- Limitation of Gini, entropy, information gain
  - Multiway split tends to give lower Gini/entropy or higher info gain
    - Therefore, {single, married, divorce} is likely to be chosen over {single+divorce, married}

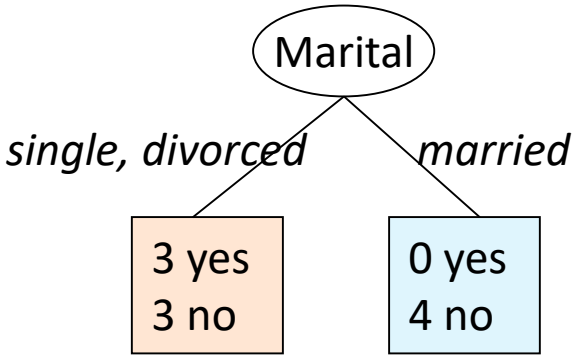
      But in this example, they give equal Gini/entropy/info gain
  - But we prefer binary split because it allows child nodes to have more data for further split
- Use **gain ratio** instead of Gini, entropy, information gain

$$Gain\ Ratio = \frac{Info\ gain}{Split\ penalty} \quad ; \quad Split\ penalty = -\sum_{all\ bins} \left(\frac{N_{bin}}{N_{root}}\right) \log_2 \left(\frac{N_{bin}}{N_{root}}\right)$$

Similar to entropy formula. So in some textbooks, it is called split info

Example : split penalty calculation





- Data at root = 10 records
- Penalty for multiway split = - 0.4 log 0.4 - 0.4 log 0.4 - 0.2 log 0.2
  = 1.522
- Penalty for binary split = - 0.6 log 0.6 - 0.4 log 0.4
  = 0.971

Pre-pruning

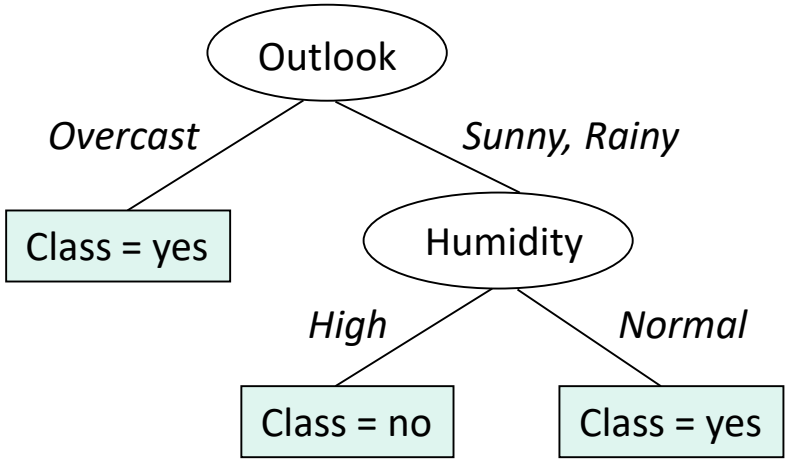
- Stop before the tree is fully grown to fit the entire data
- Some stopping conditions are
  - All records in a node belong to the same class
  - All records in a node have identical attribute values
  - Node size is too small
  - Benefit of further split is too small
  - Maximum depth is reached
- Difficult to set proper stopping conditions
  - Stopping too early may result in underfitting tree ➔ the tree doesn't learn patterns in training data
  - Stopping too late may result in overfitting tree ➔ the tree overfits training data but cannot generalize to new unseen data

Post-pruning

- Let the tree grow to its maximum size and prune it afterwards
  - Replace the whole subtree with a leaf node or a smaller subtree
  - Split data by using new root
  - Between full tree and pruned trees, choose the one with lower error
- Estimating (generalization) error
  - Optimistic approach — error = #minority records
  - Pessimistic approach — error = #minority records + penalty (more complex tree ➔ more penalty)
  - Pruning data approach — reserve a subset of training data & apply the tree to this subset
  - More details in Chapter 7

Training data for tree construction

TID	Outlook	Temperature	Humidity	Windy	Class
1	Sunny	Hot	High	False	No
2	Sunny	Hot	High	True	No
3	Overcast	Hot	High	False	Yes
4	Rainy	Mild	High	False	Yes
5	Rainy	Cold	Normal	False	Yes
6	Rainy	Cold	Normal	True	No
7	Overcast	Cold	Normal	True	Yes
8	Sunny	Mild	High	False	No
9	Sunny	Cold	Normal	False	Yes
10	Rainy	Mild	Normal	False	Yes



Training error = 2 records
= 2/10 = 0.2

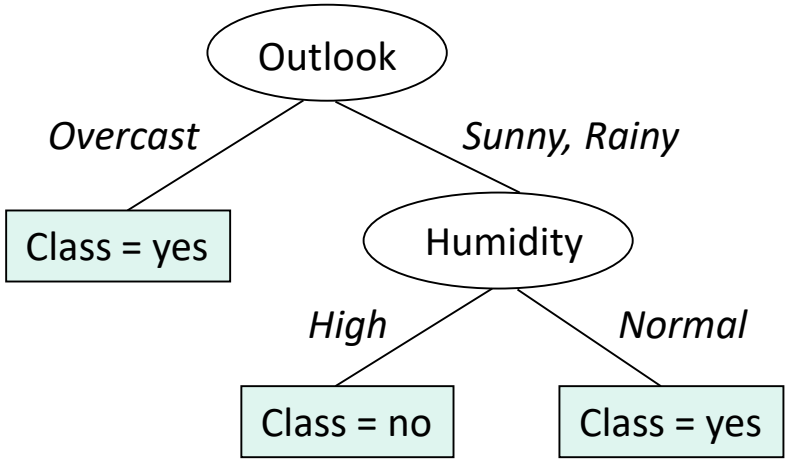
Optimistic error = 2 records
= 2/10 = 0.2

Pessimistic error = 2 + (0.5*3)
= 3.5 records
= 3.5/10 = 0.35

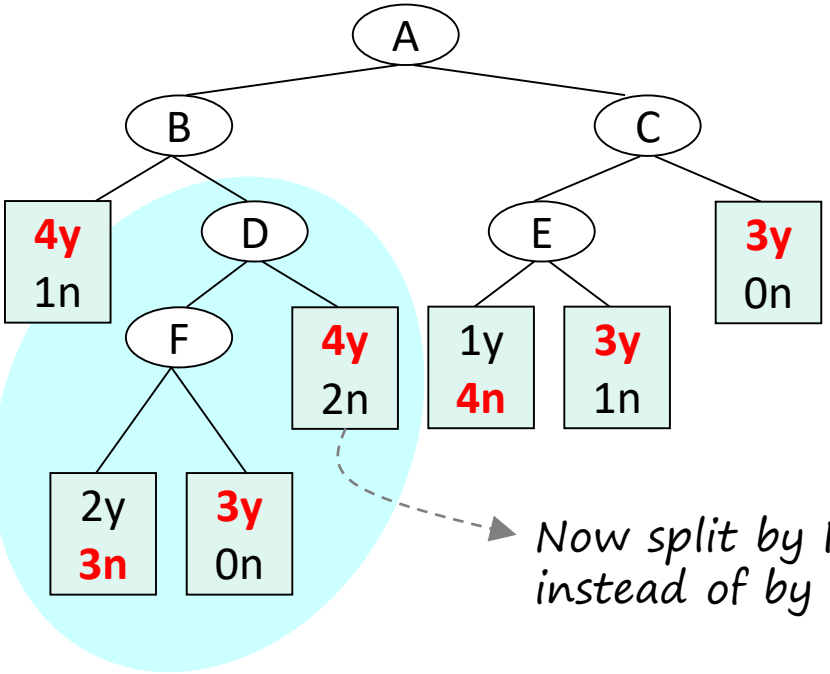
Suppose that pessimistic estimation
= training error records + 0.5 (#leafs)

Reserved training data for pruning

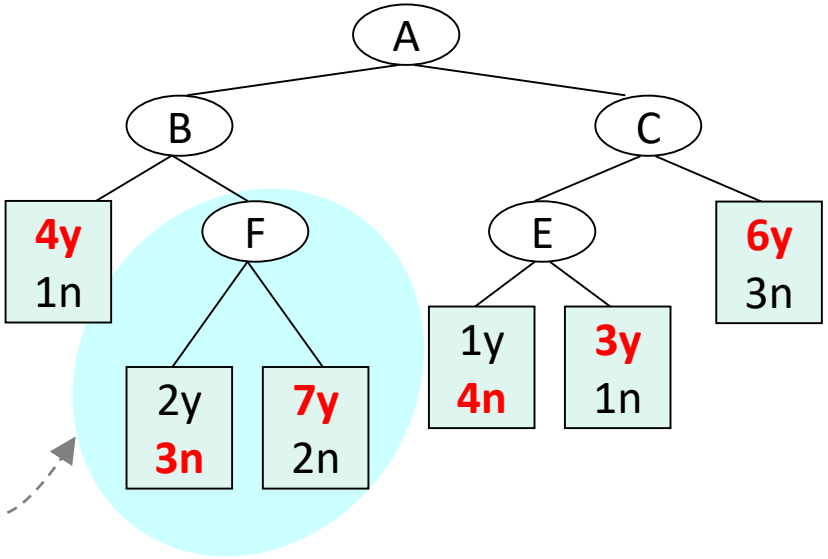
TID	Outlook	Temperature	Humidity	Windy	Class
11	Sunny	Mild	Normal	True	No
12	Overcast	Mild	High	True	Yes
13	Overcast	Hot	Normal	False	Yes
14	Rainy	Mild	High	True	No



Estimated error = 1 record
= 1/4 = 0.25



Now split by F instead of by D



Subtree's estimated error
Optimistic = 4 records
Pessimistic = 4 + (0.5 * 3) = 5.5 records

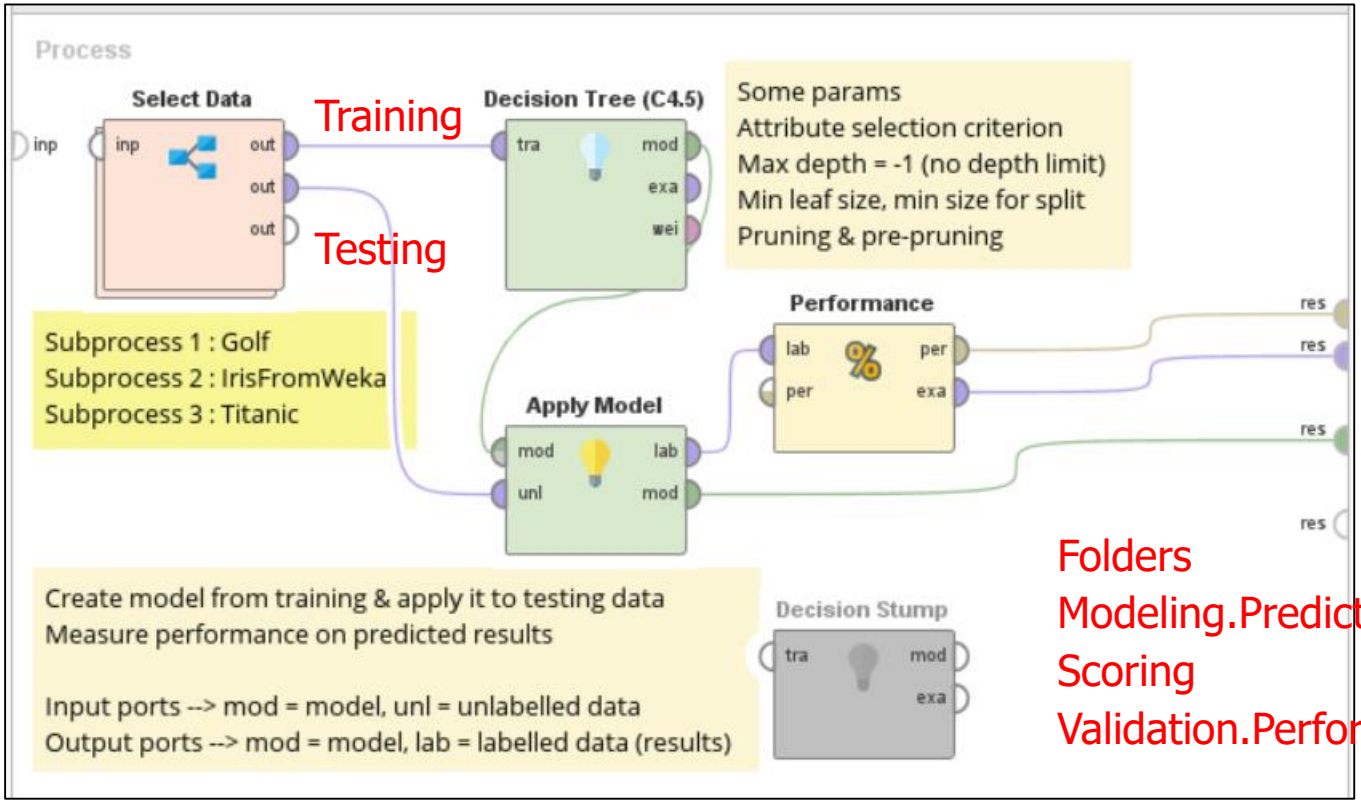
Subtree's estimated error
Optimistic = 4 records
Pessimistic = 4 + (0.5 * 2) = 5 records

Simpler subtree & same (or lower) error
So, pruning is better

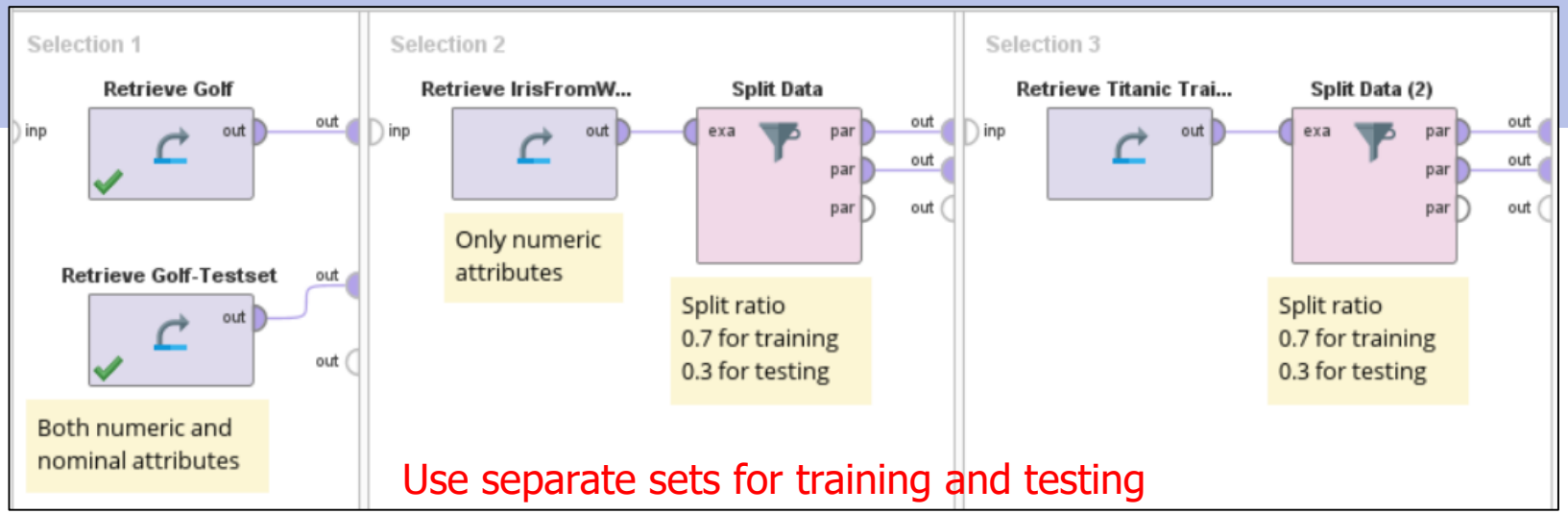
Famous Implementations

ID3	Early implementation of decision tree Support only nominal attributes The tree is not pruned
C4.5 and C5.0	Support both nominal & numeric attributes C5.0 is a commercial version of C4.5 C4.5 is called J48 in Weka
Decision stump	Create a small tree from only 1 split Typically used by ensemble methods
CHAID	Use chi-square as split criterion Support only nominal attributes

Example (1) : decision trees



Folders
Modeling.Predictive.Trees
Scoring
Validation.Performance



Use separate sets for training and testing

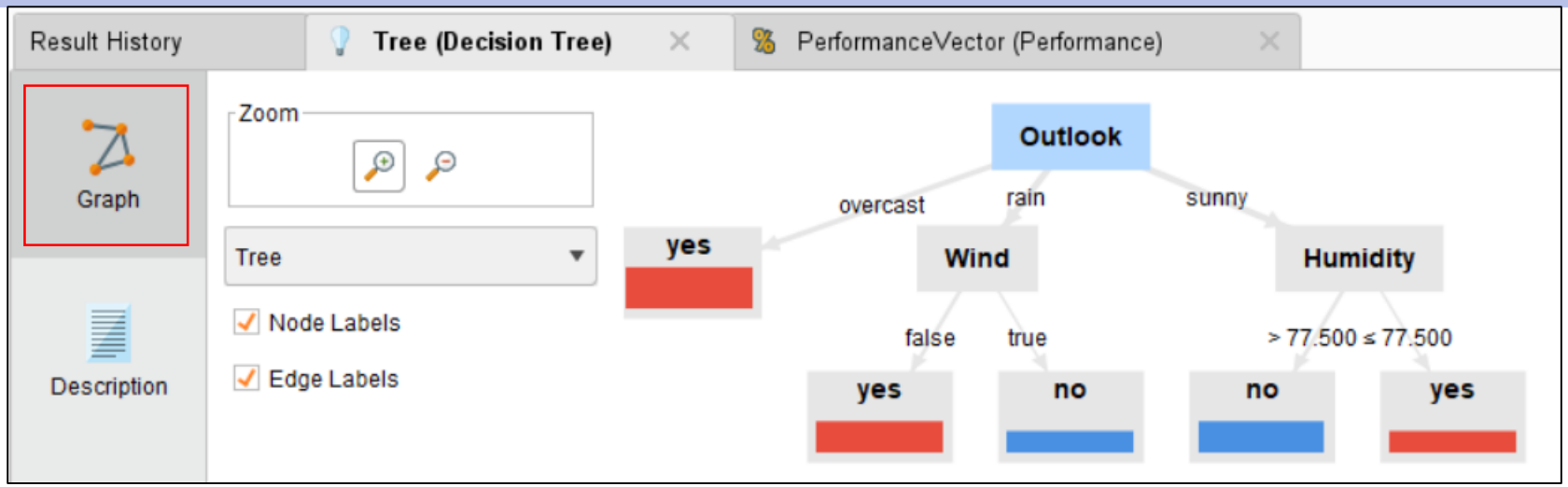
Or split one data set into training and testing

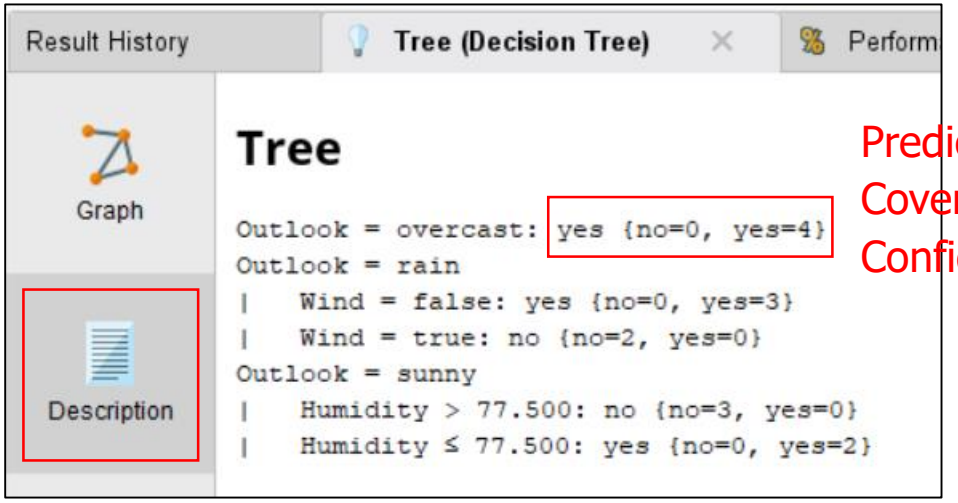
Row No.	Play	prediction(Play)	confidence(no)	confidence(yes)	Outlook	Temperature	Humidity	Wind
1	yes	no	1	0	sunny	85	85	false
2	no	yes	0	1	overcast	80	90	true
3	yes	yes	0	1	overcast	83	78	false
4	yes	yes	0	1	rain	70	96	false

After applying the model to testing data, the following attributes are created

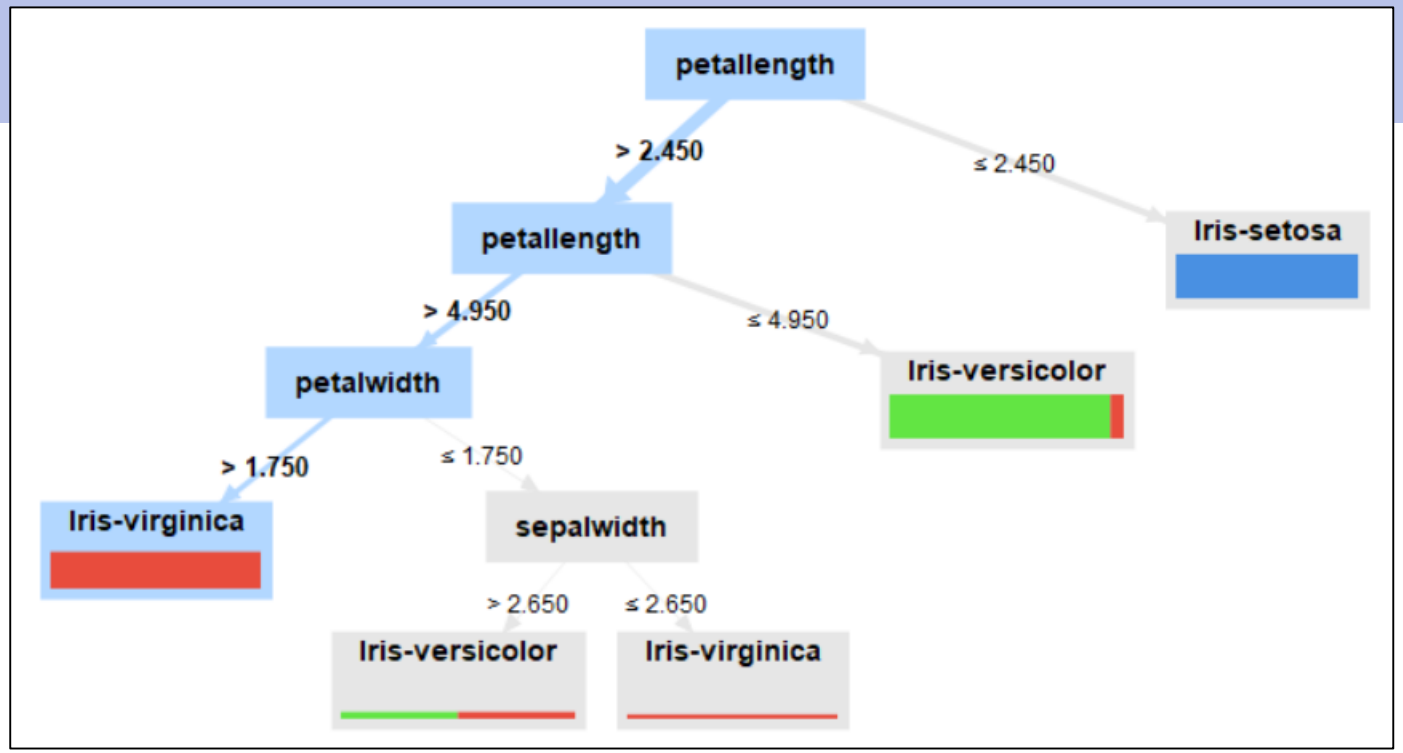
- Prediction (Play) = predicted class
- Confidence (no) = confidence (or probability or accuracy) of "no"
- Confidence (yes) = confidence (or probability or accuracy) of "yes"

Decision tree model





Prediction (i.e. majority class) = yes
Coverage = 4 records
Confidence = 4/4 = 1



When reading patterns from decision tree

- Focus on leaf nodes with high coverage (i.e. contain many records)
- Be specific about conditions along the path to a leaf node
- Prediction = majority class in that leaf node

For example, conditions for predicting Iris-Virginica are
(petal length > 4.95) and (petal width > 1.75)

Tree

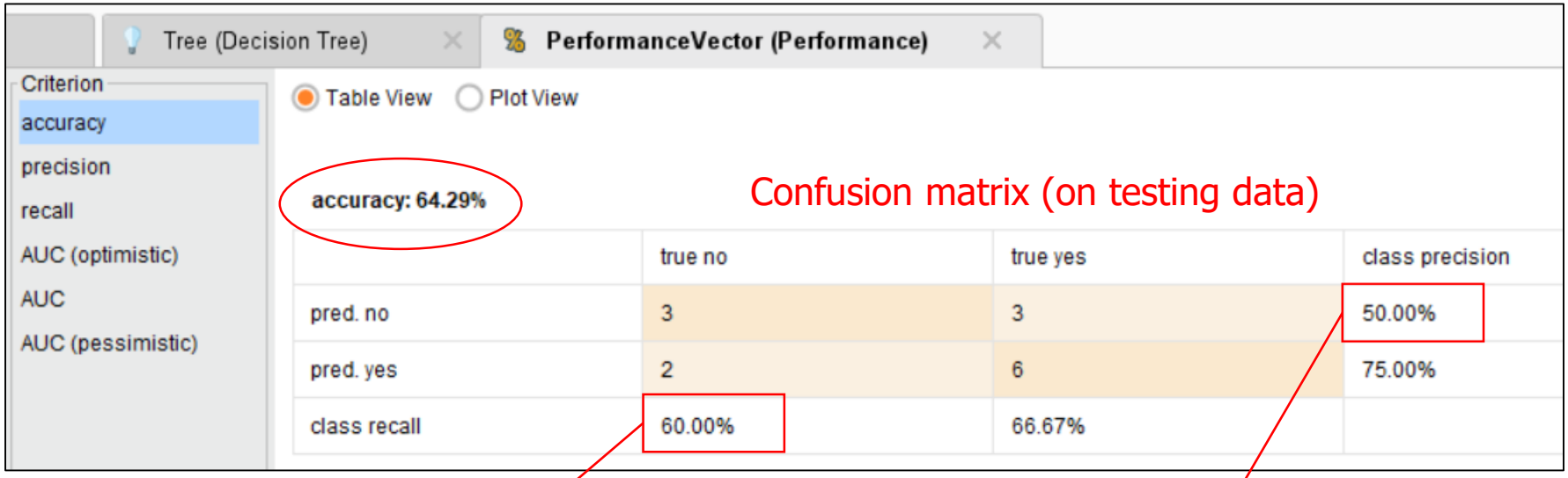
Coverage = 219 records
Confidence (yes) = 163/219 = 0.744

```
Sex = Female: Yes {Yes=163, No=56}
Sex = Male
|   No of Siblings or Spouses on Board > 4.500: No {Yes=0, No=6}
|   No of Siblings or Spouses on Board ≤ 4.500
|   |   No of Parents or Children on Board > 2.500: No {Yes=0, No=6}
|   |   No of Parents or Children on Board ≤ 2.500
|   |   |   No of Parents or Children on Board > 1.500
|   |   |   |   Age > 21: No {Yes=1, No=6}
|   |   |   |   Age ≤ 21: Yes {Yes=7, No=1}
|   |   |   No of Parents or Children on Board ≤ 1.500
|   |   |   |   No of Siblings or Spouses on Board > 2.500: No {Yes=0, No=8}
|   |   |   |   No of Siblings or Spouses on Board ≤ 2.500
|   |   |   |   |   No of Parents or Children on Board > 0.500
|   |   |   |   |   |   Age > 12: No {Yes=7, No=21}
|   |   |   |   |   |   Age ≤ 12: Yes {Yes=7, No=1}
|   |   |   |   |   No of Parents or Children on Board ≤ 0.500: No {Yes=59, No=292}
```

Row No.	Survived	prediction(S...	confidence(...	confidence(...	Sex	Age	Passenger ...
1	Yes	Yes	0.744	0.256	Female	18	First
2	No	No	0.168	0.832	Male	29.881	First
3	Yes	Yes	0.744	0.256	Female	50	First

For female passenger, we can predict "yes (i.e. survived)" with confidence = 0.744

Classification Performance

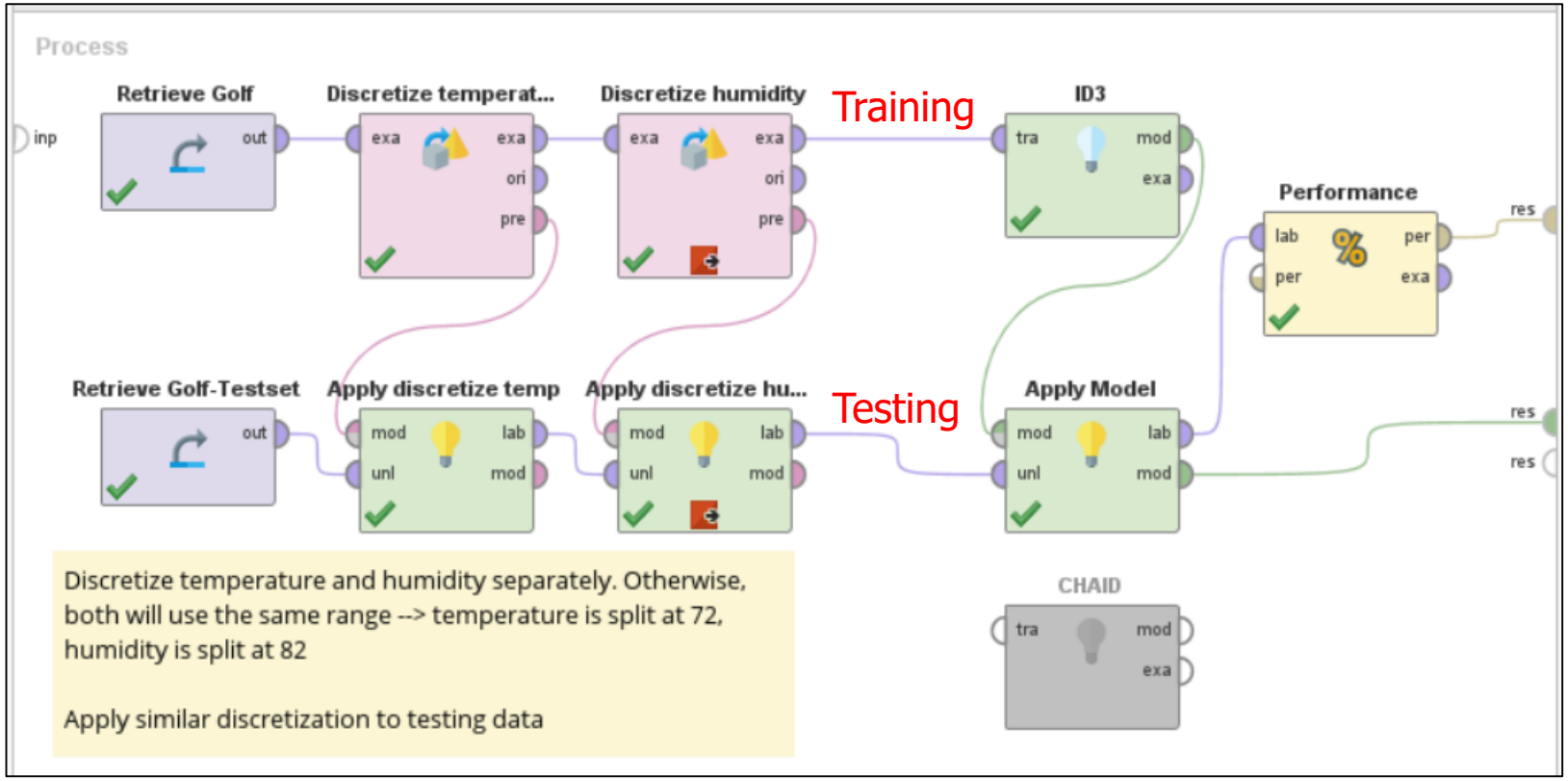


Recall (true no) = out of 5 records actually in class no, how many can be retrieved by the classifier (3/5 = 0.6)

More details in Chapter 7

Precision (pred no) = out of 6 predictions to be no, how many records are actually in class no (3/6 = 0.5)

Example (2) : decision trees for nominal data



Rule Induction

- Given a set of attributes A_1, ..., A_k
- For rule R: (precondition) $\Rightarrow$ class y
  - If precondition of R matches attributes of record X, we say

    "Rule R covers record X" OR "Record X triggers rule R"
- Suppose that
  - $|D|$ = size of data set
  - $|A|$ = number of records satisfying precondition
  - Coverage or support = $|A| / |D|$ = $P(A)$
  - Accuracy or confidence = $|A \cap y| / |A|$ = $P(y \mid A)$

Name	Blood	Give Bird	Can Fly	Live in Water	Class
Human	warm	yes	no	no	Mammals
Python	cold	no	no	no	Reptiles
Salmon	cold	no	no	yes	Fishes
Whale	warm	yes	no	yes	Mammals
Frog	cold	no	no	sometimes	Amphibians
Komodo	cold	no	no	no	Reptiles
Bat	warm	yes	yes	no	Mammals
Pigeon	warm	no	yes	no	Birds
Cat	warm	yes	no	no	Mammals
Leopard Shark	cold	yes	no	yes	Fishes
Turtle	cold	no	no	sometimes	Reptiles
Penguin	warm	no	no	sometimes	Birds
Porcupine	warm	yes	no	no	Mammals
Eel	cold	no	no	yes	Fishes
Salamander	cold	no	no	sometimes	Amphibians
Gila Monster	cold	no	no	no	Reptiles
Platypus	warm	no	no	no	Mammals
Owl	warm	no	yes	no	Birds
Dolphin	warm	yes	no	yes	Mammals
Eagle	warm	no	yes	no	Birds

Example : discovered rules

- Rule set
  - R1: (give birth = no) $\wedge$ (can fly = yes) $\Rightarrow$ Birds
  - R2: (give birth = no) $\wedge$ (live in water = yes) $\Rightarrow$ Fishes
  - R3: (give birth = yes) $\wedge$ (blood = warm) $\Rightarrow$ Mammals
  - R4: (give birth = no) $\wedge$ (can fly = no) $\Rightarrow$ Reptiles
  - R5: (live in water = sometimes) $\Rightarrow$ Amphibians

- Testing records

Name	Blood	Give Birth	Can Fly	Live in Water	Class
Hawk	warm	no	yes	no	?
Grizzly Bear	warm	yes	no	no	?
Turtle	cold	no	no	sometimes	?
Dogfish Shark	cold	yes	no	yes	?

Calculating coverage & accuracy

- Rule "R4: (give birth = no) $\wedge$ (can fly = no) $\Rightarrow$ Reptiles"
- Coverage of R4
  - Fraction of records that satisfies rule's precondition = $\mathrm{P(A)}$
  - From training data, 10 records satisfy this precondition

$$\mathrm{Coverage(R4)} = 10/20 = 50\%$$

- Accuracy of R4
  - Fraction of records (covered by the rules) that satisfies precondition and class = $\mathrm{P(y \mid A)}$
  - From 10 records covered by R4, 4 of them are reptiles

$$\mathrm{Accuracy(R4)} = 4/10 = 40\%$$

Required Properties of Rule Set

- Exhaustive
  - A rule set has exhaustive coverage if it accounts for every possible combination of attribute values
  - Otherwise, we won't be able to predict class for some records
  - Fixed by adding **default rule**
- Mutually exclusive
  - If a rule set is mutually exclusive, every possible record is covered by at most one rule ➔ in practice, a record covered by many rules is still acceptable if those rules predict the same class
  - Otherwise, there will be multiple conflicting classes for 1 record
  - Fixed by using **conflict resolution**

- R1 covers Hawk
  - Hawk triggers R1
  - So Hawk is classified as Birds
- Turtle triggers both R4 (Reptiles) and R5 (Amphibians)
  - Conflict resolution is required
- No rules are triggered by Dogfish Shark
  - Re-check the classifier
  - Add default rule = rule without any precondition

    $R_{default}$: ( ) $\Rightarrow$ $y_{default}$; $y_{default}$ = majority class

Conflict Resolution

- Rule unordering approach
  - Among all conflicted rules ➔ choose majority class
    - R1 ⇒ reptiles
    - R2 ⇒ reptiles
    - R3 ⇒ amphibians

    } predict reptiles
  - Votes may be weighted by some criteria such as accuracy
    - R1 ⇒ reptiles — accuracy = 0.3
    - R2 ⇒ reptiles — accuracy = 0.4

    } avg accuracy = 0.35
    - R3 ⇒ amphibians — accuracy = 0.9 — predict amphibians

- Rule ordering approach
  - Sort all rules in the rule set according to some criteria
  - Rule-by-rule ordering : sort rules by individual quality measures
    - Some individual measures : accuracy
    - When conflict occurs, follow the best (highest ranked) rule
  - Class-by-class ordering : group rules that predict the same classes & sort classes
    - Some class measures : #rules, total coverage from all rules
    - When conflict occurs, follow the rule in the best class

Rule-by-Rule Ordering

- When interpreting a rule, implicit meaning is assumed from the negation of the rules preceding it
  - R1: (skin cover = feather) $\wedge$ (can fly = yes) $\Rightarrow$ Birds
  - R2: (blood = warm) $\wedge$ (give birth = yes) $\Rightarrow$ Mammals
  - R3: (blood = warm) $\wedge$ (give birth = no) $\Rightarrow$ Birds
  - R4: (live in water = sometimes) $\Rightarrow$ Amphibians

  R4 is triggered ➔ A creature does not have feather, cannot fly, is cold-blooded, and sometimes lives in water, it is an Amphibian

- Ensure that a new record is predicted by the best rule
- Difficult to interpret lower-ranked rules or analyze class patterns in a large rule set

Class-by-Class Ordering

- With in group, order among rules is not important
  - R1: (skin cover = feather) $\wedge$ (can fly = yes) $\Rightarrow$ Birds
    R3: (blood = warm) $\wedge$ (give birth = no) $\Rightarrow$ Birds
  - R2: (blood = warm) $\wedge$ (give birth = yes) $\Rightarrow$ Mammals
  - R4: (live in water = sometimes) $\Rightarrow$ Amphibians

  R1 is triggered ➔ A creature has feather and can fly, it is a Bird.
  (Other attributes of birds are warm-blooded and don't give birth)

- If 1 rule in the group is triggered, the whole class information can be retrieved, allowing easier pattern analysis
- But high quality rule that predicts low-ranked class can be pushed down the order & overlooked

Rule Construction

- How to find classification rules ?
  - Indirect approach convert decision tree to rules
  - Direct approach OneR, sequential covering (RIPPER)
- Convert decision tree to rules

Tree

```
Outlook = overcast: yes {no=0, yes=4}
Outlook = rain
|   Wind = false: yes {no=0, yes=3}
|   Wind = true: no {no=2, yes=0}
Outlook = sunny
|   Humidity > 77.500: no {no=3, yes=0}
|   Humidity ≤ 77.500: yes {no=0, yes=2}
```

RuleModel (Tree to Rules) | PerformanceVector (Perfo

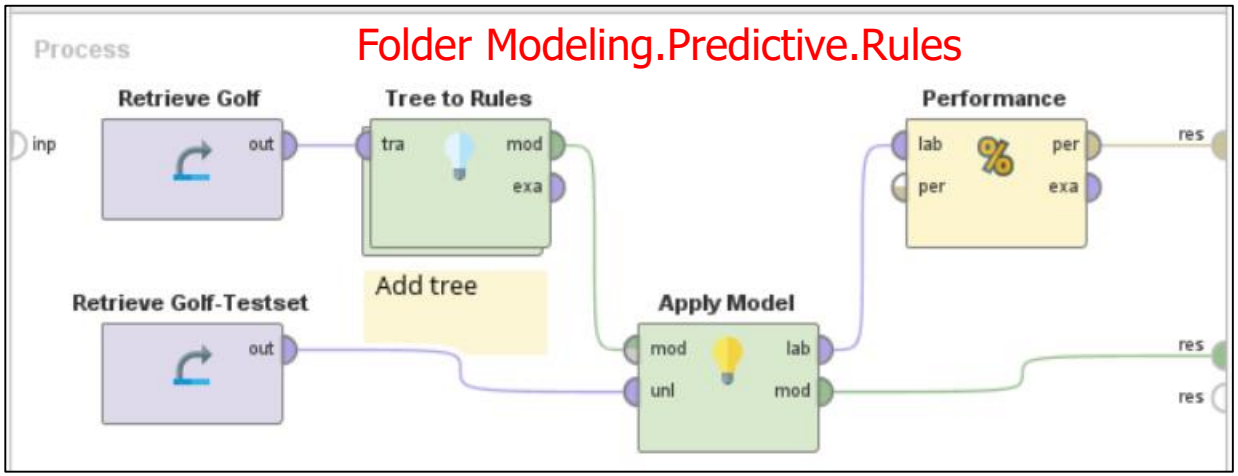
RuleModel

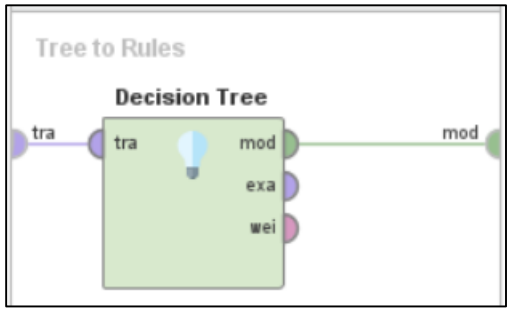
```
if Outlook = overcast then yes   (0 / 4)
if Outlook = rain and Wind = false then yes   (0 / 3)
if Outlook = rain and Wind = true then no   (2 / 0)
if Outlook = sunny and Humidity > 77.500 then no   (3 / 0)
if Outlook = sunny and Humidity ≤ 77.500 then yes   (0 / 2)

correct: 14 out of 14 training examples.
```

Annotations

Example (3) : tree to rules





OneR (Single Rule Induction)

- Extract rules directly from training data

For each attribute i {

- If it is nominal attribute with n categories ➔ split records into n bins and generate rules

  if (attribute = category j), prediction = majority class in bin j

- If it is numeric attribute ➔ find best splits from discretization ➔ split records into n bins and generate rules

  if (attribute is in range j), prediction = majority class in bin j

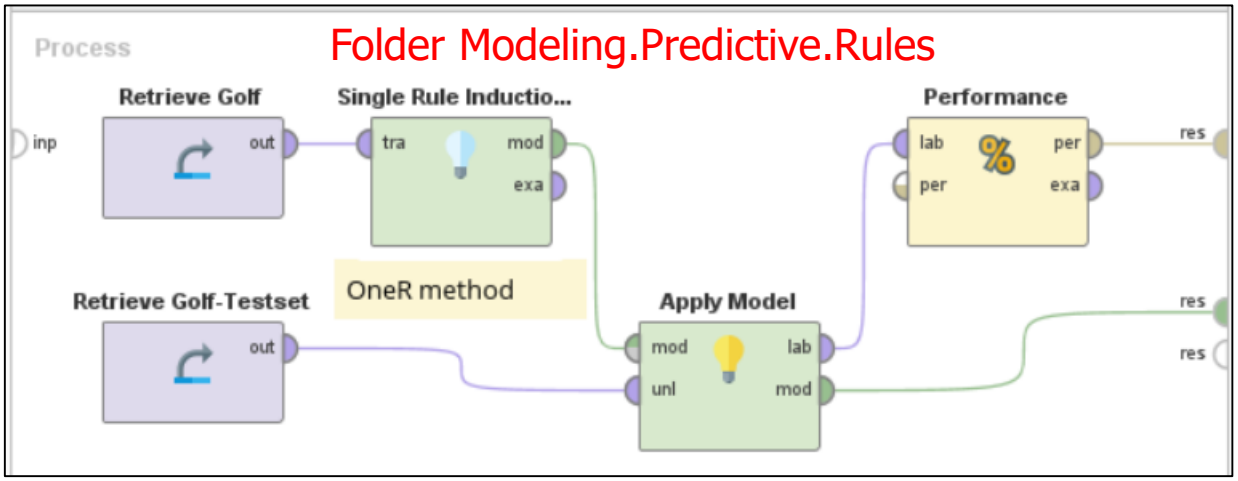
- Rule set i has n rules for n bins

}

Choose only 1 rule set with lowest training error

Error when using model to predict training data (= total minority records in all bins)

Example (4) : single rule induction (OneR)



RuleModel

```
if Temperature ≤ 64.500 then yes  (0 / 1)
if Temperature ≤ 66.500 then no   (1 / 0)
if Temperature ≤ 70.500 then yes  (0 / 3)
if Temperature ≤ 73.500 then no   (2 / 1)
if Temperature ≤ 77.500 then yes  (0 / 2)
if Temperature ≤ 80.500 then no   (1 / 0)
if Temperature ≤ 84 then yes  (0 / 2)
else no  (1 / 0)

correct: 13 out of 14 training examples.
```

Rule set based on temperature gives the lowest training error

So we use only temperature as the predictor in OneR rules

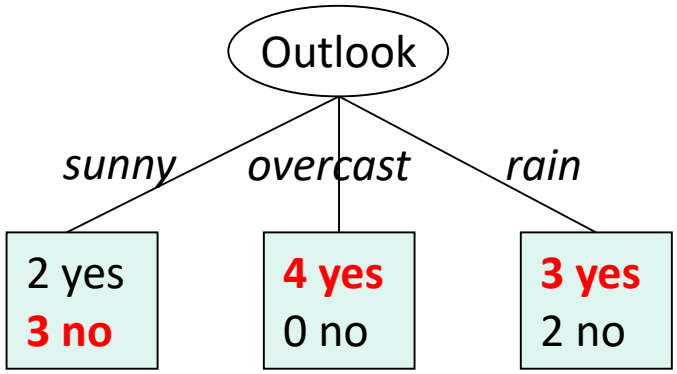
Row No.	Play	Outlook	Temperature	Humidity	Wind
1	no	sunny	85	85	false
2	no	sunny	80	90	true
3	yes	overcast	83	78	false
4	yes	rain	70	96	false
5	yes	rain	68	80	false
6	no	rain	65	70	true
7	yes	overcast	64	65	true
8	no	sunny	72	95	false
9	yes	sunny	69	70	false
10	yes	rain	75	80	false
11	yes	sunny	75	70	true
12	yes	overcast	72	90	true
13	yes	overcast	81	75	false
14	no	rain	71	80	true

Compare training errors from 4 rule sets
1. Outlook (3 rules)
2. Wind (2 rules)
3. Temperature
4. Humidity

Number of rules for numeric attributes depends on discretization

Rule set based on outlook
1. if (outlook = sunny), then no
2. if (outlook = overcast), then yes
3. if (outlook = rain), then yes

Training error = 4/14 = 28.57%



Sequential Covering Method

- A popular implementation is RIPPER (Repeated Incremental Pruning to Produce Error Reduction)
- Extract rules for each class, starting from smaller to bigger classes
  - E.g. in Golf data set, there are 9 yes + 5 no
  - Find rules for class no first

For each class {

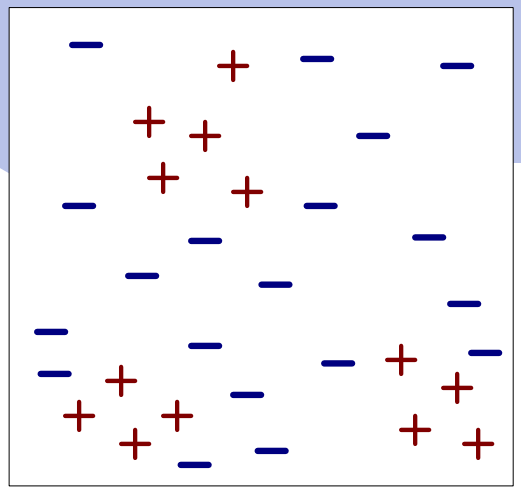
1. Grow a new rule from remaining records
2. Remove records covered by current rule before growing the next one

}

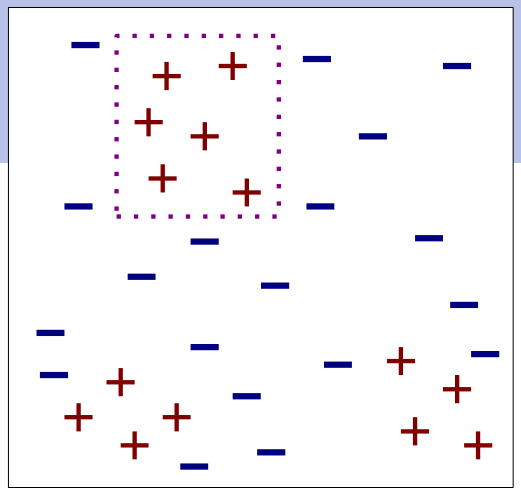
Pruning can be applied to get simpler rules

Add default rule : if ( ), prediction = biggest class

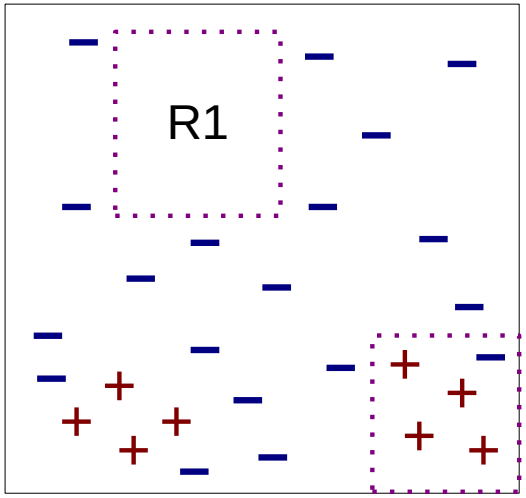
Sort classes, starting from the biggest ones



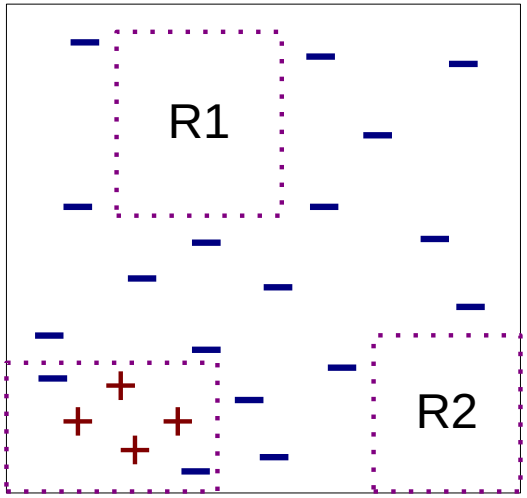
Original data



Step 1



Step 2



Step 3

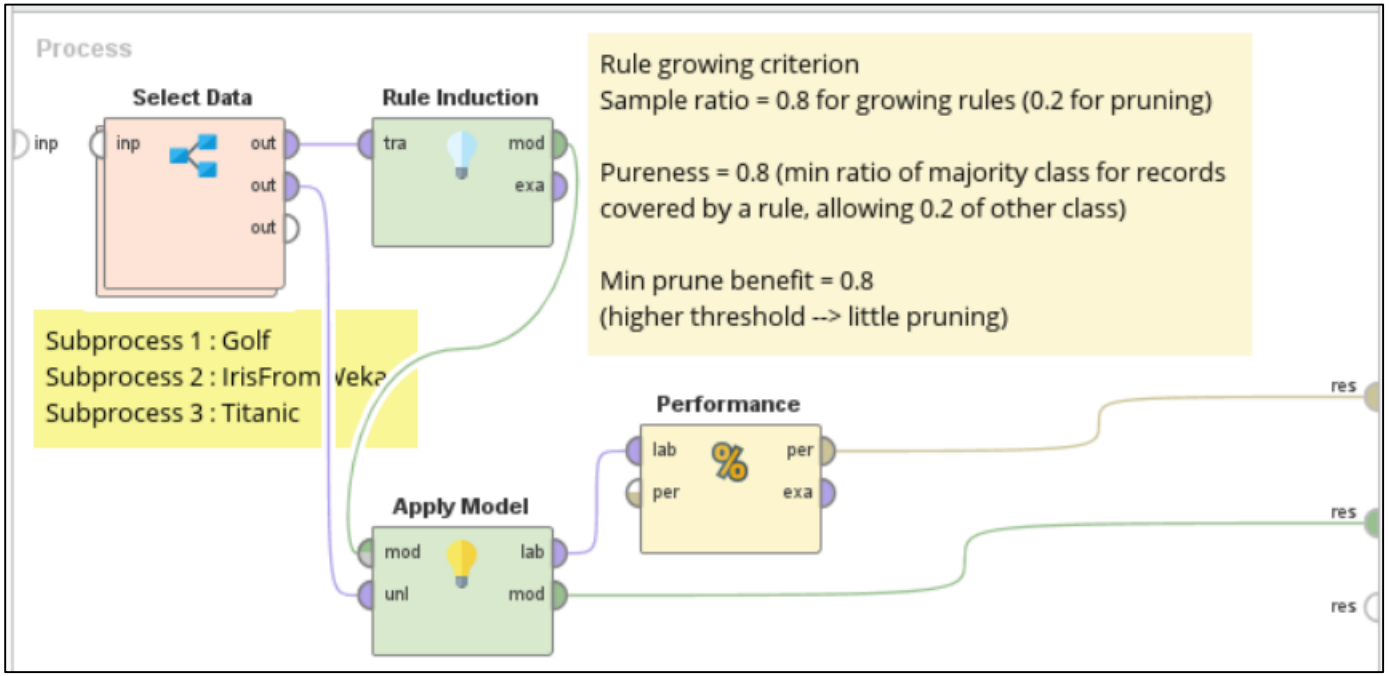
3 rules are extracted for class +

Rule Growing Approaches

- General to specific. RIPPER uses this approach
  - Starting from general rule : if ( ), prediction = no
  - Without any condition, the rule covers all 14 records but only 5 are actually in class no. Accuracy = 5/14 or training error = 9/14
  - Keep adding conditions to improve accuracy or reduce training error
  - RIPPER uses information gain instead of accuracy
- Specific to general
  - Random a record & make a specific rule from that record
  - Rule from record 1 : if (outlook = sunny) and (temperature in 85's bin) and (humidity in 85's bin) and (wind = false), prediction = no
  - Accuracy = 1/1 or training error = 0/1
  - But it has poor coverage ➔ keep removing conditions to improve coverage but accuracy must not fall below a threshold

- After getting each rule ➔ remove all records covered by that rule before growing the next one
  - Recall that a rule covers a record if all attributes in that record match the conditions in the rule
  - E.g. rule if (outlook = sunny) and (wind = false), prediction = no covers records 1, 8, 9 ➔ remove them from data set
    - Records 1 and 8 are in class no; record 9 is in class yes
    - We don't care about class labels, only attributes
  - *This is why we extract rules for smaller class first. Doing the opposite will cause too many records to be removed in early stages*
- If a subset of training data is reserved for pruning, apply complete and pruned rules to this subset ➔ choose the option that gives more benefit e.g. better accuracy

Example (5) : RIPPER



RuleModel

```
if Sex = Male then No  (81 / 341)
if Passenger Class = First then Yes  (61 / 3)
if Passenger Class = Second then Yes  (49 / 9)
if Passenger Fare ≤ 24.808 and Passenger Fare ≤ 7.742 then Yes  (11 / 1)
if Passenger Fare > 24.808 then No  (0 / 14)
if Passenger Fare > 15.700 and Age > 21 then Yes  (7 / 0)
if Age ≤ 27.500 and Passenger Fare ≤ 8.590 then Yes  (11 / 2)
if Age > 22.500 and Passenger Fare > 8.271 and Age ≤ 26.500 then No  (0 / 3)
if Age ≤ 29.941 and Passenger Fare ≤ 14.177 and Passenger Fare > 9.840 then Yes  (7 / 1)
if Passenger Fare > 9.706 and Passenger Fare ≤ 15.373 then No  (1 / 8)
if Age ≤ 29.941 and No of Siblings or Spouses on Board ≤ 2.500 then Yes  (15 / 6)
else No  (0 / 7)

correct: 534 out of 638 training examples.
```

Confidence (no) = 0.808

Confidence (yes) = 0.845

Default rule (majority class = no), triggered when conditions in all preceding rules are not matched

RapidMiner's rules are not grouped by classes

When predicting each testing record ➔ follow the first triggered rule

Row No.	Survived	prediction...	confidence(Yes)	confidence(No)	Age	Pass...	Sex	No of Sib...	No of Par...	Passenger Fare
1	Yes	Yes	0.953	0.047	18	First	Female	1	0	227.525
2	No	No	0.192	0.808	29.881	First	Male	0	0	25.925
3	Yes	Yes	0.953	0.047	50	First	Female	0	1	247.521
112	Yes	Yes	0.845	0.155	17	Second	Female	0	0	12
113	Yes	Yes	0.845	0.155	34	Second	Female	0	0	10.500
114	No	No	0.192	0.808	36	Second	Male	0	0	12.875

Record 2 first triggered rule = rule 1 confidence (no) = 0.808

Record 112 first triggered rule = rule 3 confidence (yes) = 0.845

Summary

- Both decision trees and rules are expressive & easy to interpret
- Computationally inexpensive
  - Can handle large training data. Training is fast
  - Predicting a new record is also fast
- Both are based on greedy selection
  - Consider attributes individually
  - Decision tree chooses attribute that best splits data to purer nodes
  - RIPPER chooses attribute to improve current rule
- Important attributes are eventually chosen to be tree nodes or rule conditions. Dominant attributes are chosen in early stages

- Interaction between attributes are somewhat captured
  - Later chosen attributes help previous ones improve the quality of splitting or predicting
  - But much & crucial interaction between attributes can be missing, compared to other methods that model all attributes at once (e.g. neural network)
- RIPPER is better than DT at handling imbalance classes
  - Rules for smaller class are extracted first